

Customer Churn Prediction

Data Engineering Report

Preprocessing Pipeline — Full Documentation

Role: Data Engineer

Project: Customer Churn Prediction

Dataset: customer_churn.csv | 999,999 Rows | 32 Columns

1. Project Overview

This report documents the complete Data Engineering pipeline applied to the Customer Churn dataset. The goal is to prepare clean, model-ready data by handling missing values, encoding categorical features, scaling numerical features, and splitting the data for training and testing.

Target Variable: churn (0 = Customer Stayed, 1 = Customer Left)

Metric	Value
Total Rows	999,999
Total Columns	32
Churn Rate	9.9% (99,227 customers)
Duplicate Rows	0
Columns with Missing Values	5

2. Raw Dataset — Column Overview

The dataset contains 32 columns covering customer demographics, financial data, service subscriptions, behavioral metrics, and risk indicators.

Column	Type	Description
customer_id	ID	Unique customer identifier — dropped
signup_date	Date	Registration date — dropped (tenure used instead)
age	Numeric	Customer age (18–90)
gender	Categorical	Male / Female / Other
annual_income	Numeric	Annual income — 3% missing
education	Ordinal	high_school / college / bachelor / master / phd
marital_status	Categorical	married / single / divorced / widowed
tenure	Numeric	Months as customer (key churn indicator)
contract	Ordinal	month_to_month / one_year / two_year
monthlycharges	Numeric	Monthly bill amount
totalcharges	Numeric	Total amount paid
payment_method	Categorical	4 payment types
paperless_billing	Binary	Yes / No

Column	Type	Description
num_complaints	Numeric	Number of complaints — 3% missing
customer_satisfaction	Numeric	Satisfaction score 1–9 — 2% missing
avg_monthly_gb	Numeric	Average monthly data usage — 5% missing
credit_score	Numeric	Credit score 300–850 — 4% missing
churn	Target	0 = Stayed, 1 = Left

3. Missing Values Analysis

Five columns contained missing values. All were below 5% threshold, making them safe to impute rather than drop.

Column	Missing Count	Missing %	Strategy
avg_monthly_gb	50,012	5.0%	Fill with Median
credit_score	40,394	4.0%	Fill with Median
annual_income	29,959	3.0%	Fill with Median
num_complaints	29,906	3.0%	Fill with Median
customer_satisfaction	19,921	2.0%	Fill with Median

Why Median and not Mean?

Median is resistant to outliers. For example, annual_income may contain very high values that would skew the mean significantly. The median represents the true middle value regardless of extreme values, making it a more reliable imputation strategy.

Python code used:

```
missing_cols = ['annual_income', 'customer_satisfaction',  
                'num_complaints', 'avg_monthly_gb', 'credit_score']  
for col in missing_cols:  
    df[col] = df[col].fillna(df[col].median())
```

4. Dropped Columns

Two columns were removed as they do not contribute to model learning:

Column	Reason for Dropping
customer_id	Unique identifier per row — no predictive value
signup_date	tenure column already captures customer duration

5. Feature Encoding

Categorical columns were converted to numbers using two strategies depending on whether the values have a natural order.

5.1 Ordinal Encoding — Columns with Natural Order

Used when one value is logically greater than another.

Column	Mapping
paperless_billing	No = 0 Yes = 1
education	high_school=0 college=1 bachelor=2 master=3 phd=4
contract	month_to_month=0 one_year=1 two_year=2

5.2 One-Hot Encoding — Columns without Order

Used when values have no ranking. Each unique value becomes a separate binary column (0 or 1).

Column	Unique Values	New Columns Created
gender	Male, Female, Other	gender_Male, gender_Female, gender_Other
marital_status	married, single, divorced, widowed	4 binary columns
payment_method	4 payment types	4 binary columns

Result after encoding: 32 columns → 38 columns

6. Feature Scaling

Numerical columns operate on different scales. Without scaling, models may over-prioritize large-magnitude features. StandardScaler was applied to bring all features to the same scale (mean=0, std=1).

Columns Scaled

Column	Original Range	After Scaling
age	18 – 90	mean $\approx$ 0, std $\approx$ 1
annual_income	~\$10k – \$200k	mean $\approx$ 0, std $\approx$ 1
tenure	0 – 72 months	mean $\approx$ 0, std $\approx$ 1
monthlycharges	\$18 – \$855	mean $\approx$ 0, std $\approx$ 1
totalcharges	Varies widely	mean $\approx$ 0, std $\approx$ 1
avg_monthly_gb	Usage data	mean $\approx$ 0, std $\approx$ 1
credit_score	300 – 850	mean $\approx$ 0, std $\approx$ 1
num_complaints	0 – 7	mean $\approx$ 0, std $\approx$ 1
num_service_calls	Count	mean $\approx$ 0, std $\approx$ 1
customer_satisfaction	1 – 9	mean $\approx$ 0, std $\approx$ 1
days_since_last_interaction	Days	mean $\approx$ 0, std $\approx$ 1
late_payments	Count	mean $\approx$ 0, std $\approx$ 1
num_services	0 – 6	mean $\approx$ 0, std $\approx$ 1

Important Note — Data Leakage Prevention:

StandardScaler was fit only on training data (X_train). The same scaler was then used to transform X_test. This prevents any information from the test set leaking into the training process.

7. Train / Test Split

The cleaned dataset was split into training (80%) and testing (20%) sets.

Split	Rows	Churn Rate
Training Set	799,999	9.9%
Test Set	200,000	9.9%
Total	999,999	9.9%

stratify=y parameter was used to ensure the churn rate (9.9%) is preserved equally in both training and test sets. Without stratify, random splitting could result in imbalanced subsets.

Class Imbalance Note

The dataset has a significant class imbalance: 90.1% No Churn vs 9.9% Churn. This was addressed in the modeling phase using class_weight='balanced' parameter in the models, which adjusts the importance of each class inversely proportional to its frequency.

8. Preprocessing Pipeline Summary

Step	Action	Result
1	Load raw data	999,999 rows × 32 columns
2	Drop useless columns	Removed customer_id, signup_date
3	Handle missing values	5 columns filled with Median — 0 nulls remaining
4	Ordinal Encoding	education, contract, paperless_billing → integers
5	One-Hot Encoding	gender, marital_status, payment_method → binary cols
6	Feature Scaling	13 numerical columns → StandardScaler applied
7	Train/Test Split	80% Train (799,999) / 20% Test (200,000)

9. Deliverables

File	Location	Description
churn_cleaned.csv	data/processed/	Final clean dataset ready for modeling
scaler.pkl	models/	Fitted StandardScaler for deployment use
02_data_preprocessing.ipynb	notebooks/	Full preprocessing notebook with code
This Report	reports/	Data Engineering documentation